Programming Assignment I - Search Algorithms

Deniz K. Acikbas | CIS 579 - Introduction to AI | October 9th 2026

Initial board

-	2	3
1	4	5
8	7	6

Goal board

1	2	3
8	-	4
7	6	5

Experiment setup

Eight required search configurations use the same initial and goal boards. Each blank move costs 1. Both heuristics ignore the blank.

All returned action sequences were replayed and verified to reach the goal.

Comparison of search results

Metric	BFS	DFS	IDS	UCS	GBS Misplaced	GBS Manhattan	A* Misplaced	A* Manhattan
Path cost	6	26,608	6	6	6	6	6	6
Solution depth (moves)	6	26,608	6	6	6	6	6	6
Search depth (max. generated)	7	66,126	6	7	6	6	6	6
Search time (seconds)	0.000711	2.455654	0.001431	0.000908	0.000108	0.000131	0.000136	0.000170
Nodes expanded	58	159,413	104	58	6	6	6	6
States explored	59	159,414	188	59	7	7	7	7
Unique states discovered	97	179,773	84	97	11	11	11	11

Source: one fresh run per configuration on Linux using Python 3.14.4. Times exclude board display and report writing; short timings vary between runs. IDS includes repeated visits across depth limits.

Shared six-move solution: DOWN, DOWN, RIGHT, RIGHT, UP, LEFT. All configurations except DFS returned this shortest path.

Findings and measurement notes

Interpretation of the results

BFS and UCS: both found a path of cost 6 and expanded 58 nodes. Since every move costs 1, increasing path cost follows the same order as increasing depth with these implementations and tie-breaking rules.

IDS: found the same six-move solution, but expanded 104 nodes across repeated depth limits. Its maximum generated depth was 6; its unique discovered-state count was 84.

DFS: reached the goal after a path of 26,608 moves, with 159,413 expansions and a maximum generated depth of 66,126. These are depths of the discovered search paths, not shortest distances between puzzle states. The stack follows long branches determined by successor order, so DFS does not guarantee a shortest solution.

GBS and A*: each required heuristic configuration returned the six-move path with 6 expansions and 11 discovered states. Both initial heuristic values are 6 for this particular board. These equal results on one easy instance do not establish that the heuristics perform equally on harder puzzles.

Optimality: BFS establishes that the shortest path for this instance has cost 6. UCS, IDS, and A* also return that path. GBS happens to find it here, but is not generally optimal. A* combines path cost with the misplaced-tile or Manhattan estimate; both estimates are admissible for this unit-cost puzzle.

How measurements were recorded

Path cost / solution depth: number of actions in the returned path, because each action costs 1. **Search depth:** largest depth of any generated search node. BFS and UCS generated depth-7 nodes before removing the depth-6 goal from their frontiers.

Expanded nodes: nodes for which successors were generated. **Explored states:** nodes checked for the goal. **Discovered states:** distinct boards encountered, including queued boards. Expansion counts describe search work; they do not directly measure memory in bytes. IDS revisits boards at successive limits, so its visit and expansion totals include repetitions.

Timing: each algorithm used **perf_counter()** with step display disabled. Values are shown in seconds to six decimal places. These are single-run observations, not averages or a general performance ranking. Full paths and measurements are saved locally in Code/results/output_*.txt. Optional Beam search and additional heuristics are not implemented.

References and acknowledgment

CIS 579 Programming Assignment 1, pages 2-3: required algorithms, output fields, and comparison table. UC Berkeley Pacman AI Projects (http://ai.berkeley.edu): supplied starter code, with attribution retained. ChatGPT, OpenAI: assistance with implementation, explanations, debugging, verification, and report preparation.